

Autonomous FinOps Engine

Project Architecture & Development Blueprint | Shaurya Malhan

1. The Core Problem & The Mission

In 2026, global enterprises waste roughly 30% of their annual cloud budget (billions of dollars) on idle compute, zombie Kubernetes pods, and over-provisioned servers. Developers over-allocate CPU and RAM requests "just in case" of traffic spikes, leading to massive financial waste.

The Mission: Build an Autonomous FinOps Agent driven by eBPF. It provides zero-instrumentation, mathematically undeniable hardware visibility with near-zero performance overhead, and uses that data to automatically resize cloud infrastructure.

Flaws in the Current Market:

- **Generation 1 (AWS Cost Explorer):** Relies on developers manually tagging resources. If they forget, the cost becomes untraceable.
- **Generation 2 (Datadog, Kubecost):** Runs heavy user-space agents that consume 2-5% of the server's CPU. Relies on Kubernetes API requests rather than actual physical hardware usage.
- **Generation 3 (ScaleOps):** Actively resizes pods, but lacks the granular, network-level financial reporting required by finance teams.

2. High-Level Architecture (HLD) & System Flow

This project fundamentally re-engineers how monitoring works. Instead of sitting *on top* of the operating system, the platform sits *inside* it.

Step 1: The Kernel Layer (eBPF Probes)

Written in C++ / Rust

Compile eBPF programs into bytecode and attach them to Linux kernel hooks (like `sys_enter`). Every time a container asks the processor to compute data or asks RAM to store data, the probe records the exact cycle count and byte allocation. Extracts absolute hardware truth without modifying application code.

Step 2: The Local Data Agent

Written in Rust

A lightweight, lock-free Rust daemon running in user space. It reads the raw metrics from the kernel via eBPF ring buffers. It aggregates and compresses millions of micro-events into batched payloads. Uses pre-allocated memory pools (no dynamic malloc) to consume less than 0.1% CPU.

Step 3: The Ingestion Engine

Node.js, Kafka, AWS ECS

Thousands of Rust agents stream compressed payloads via gRPC to the AWS backend. A fleet of Dockerized Node.js load balancers receives the data and pushes it into Apache Kafka, ensuring databases don't crash during traffic spikes.

Step 4: The TSDB & Financial Translation

ClickHouse or TimescaleDB

Store metrics in a Time-Series Database (TSDB). A background worker continuously joins hardware metrics with live AWS/GCP pricing APIs to calculate the exact dollar amount wasted per microservice per hour.

Step 5: The Control Plane & UI

React & Kubernetes APIs

The frontend visualizes financial waste and features an automated remediation engine. When it identifies wasted RAM, it automatically generates a Terraform script or a Kubernetes manifest PR to safely scale down over-provisioned pods.

3. The 24-Month Roadmap (Learn & Build)

Bridge the gap between competitive programming logic and production system design. System design is learned by actively solving bottlenecks.

Months 1-3

Phase 1: OS Internals & The Sandbox

- **Research:** Linux Virtual Memory (Page Tables), the Completely Fair Scheduler (CFS), and the anatomy of a System Call (LLD).
- **Build:** Setup a Linux VM. Install Clang, LLVM, and the Rust toolchain. Write a "Hello World" eBPF kprobe intercepting process execution.

Months 4-8

Phase 2: High-Performance Data Streaming

- **Research:** Concurrency, asynchronous runtimes (Tokio), and lock-free data structures (Ring Buffers, Skip Lists).
- **Build:** Architect the Rust user-space agent to read from the eBPF map, aggregate data, and batch for network transit using O(1) memory allocation.

Months 9-15

Phase 3: The Distributed Backend (HLD)

- **Research:** Microservice architectures, gRPC networking, message brokers (Kafka), and Time-Series Databases (HLD).
- **Build:** Containerize a Node.js backend using Docker on AWS ECS. Create gRPC endpoints, configure Kafka, and write data to TimescaleDB.

Months 16-20

Phase 4: Financial Analytics & Control Plane

- **Research:** Cloud Provider Pricing APIs, advanced SQL aggregations, and Infrastructure as Code (Terraform).
- **Build:** Build the React frontend. Design mathematical logic correlating exact CPU usage with live AWS pricing to output a dollar amount.

Months 21-24

Phase 5: Automated Remediation (The Moat)

- **Research:** Kubernetes API Server, safe rollback strategies, and distributed consensus.
- **Build:** Implement the backend workers communicating directly with Kubernetes clusters to dynamically adjust resources safely.

4. Scalability Vectors & Enterprise Target Markets

Target Market 1: AI Inference Startups

The Problem: Burning millions on A100/H100 GPUs, over-provisioning clusters due to variable LLM traffic.

How You Scale: Extend eBPF probes to track GPU memory allocation and Tensor Core wait times at the kernel driver level.

Why You Win: Standard tools cannot see deep into GPU drivers efficiently.

Target Market 2: High-Frequency Trading (HFT) & Gaming

The Problem: Refusal to install monitoring tools because 2% CPU overhead ruins microsecond latency.

How You Scale: Emphasize the zero-allocation nature of the Rust/eBPF agent bypassing standard memory allocation.

Why You Win: Full financial visibility with literally $O(1)$ overhead on their critical path.

Target Market 3: Standard Enterprise SaaS

The Problem: Massive sprawling architectures with thousands of untracked web microservices.

How You Scale: Kafka + TSDB distributed ingestion engine scales horizontally across AWS ECS.

Why You Win: Consolidation. Complete visibility and automated remediation in one tool.